

LA CONQUISTA

Memoria ampliada del Proyecto Final

Metodología de la Programación y Estructuras de Datos

Grado en Matemáticas y Computación - 1.º curso

Universidad de Alcalá - Grupo H08G05

Mayo de 2026

Integrante	Rol	Responsabilidad principal
Marcos Fernández Pérez	Persona 1	Metodología, requisitos, UML, memoria y pruebas de dominio
Asier López Rodero	Persona 2	Estructuras de datos propias, algoritmos y pruebas de estructuras
Mateo Ramos Gómez	Persona 3	Lógica de juego, JavaFX, persistencia JSON e integración

Índice

1. Introducción y objetivos
2. Requisitos y alcance final
3. Metodología y reparto de trabajo
4. Arquitectura del proyecto
5. Modelo de dominio y reglas de juego
6. Estructuras de datos propias y costes
7. Algoritmos principales
8. Persistencia JSON
9. Interfaz JavaFX
10. Casos de uso
11. Invariantes y contratos
12. UML
13. Pruebas realizadas
14. Uso de IA
15. Crítica del proyecto
16. Ejecución y entrega

1. Introducción y objetivos

La Conquista es un juego por turnos de tipo dungeon crawler desarrollado en Java y JavaFX. El jugador explora una campaña formada por habitaciones conectadas mediante un grafo, recoge objetos, combate enemigos, evita trampas y debe alcanzar la salida final antes de quedarse sin vida o sin turnos.

El trabajo combina los contenidos de Metodología de la Programación y Estructuras de Datos: análisis de requisitos, reparto de responsabilidades, diseño orientado a objetos, estructuras propias, pruebas, persistencia, interfaz gráfica y uso documentado de herramientas de IA.

- Objetivo funcional: construir una partida jugable con exploración, combate, inventario, puertas, trampas, ruta mínima, guardado y carga.
- Objetivo técnico: demostrar el uso de estructuras de datos propias y razonar sus costes de operación.
- Objetivo metodológico: mantener trazabilidad entre requisitos, decisiones, UML, pruebas y memoria final.
- Objetivo de calidad: entregar un sistema defendible, ejecutable y explicable por todos los miembros del grupo.

2. Requisitos y alcance final

El alcance final se ha definido a partir del enunciado y de las decisiones cerradas durante la implementación. La versión final incluye campaña, interfaz JavaFX, persistencia JSON, ruta mínima, turnos, pruebas y estructuras propias.

ID	Requisito funcional	Estado
RF-01	Iniciar nueva partida desde configuración JSON	Implementado
RF-02	Mostrar la habitación actual como matriz de celdas	Implementado
RF-03	Mover al jugador de forma ortogonal y validar obstáculos	Implementado
RF-04	Gestionar turnos: jugador primero y enemigos después por orden	Implementado
RF-05	Permitir como máximo un movimiento y una acción por turno	Implementado
RF-06	Mover enemigos hacia el jugador y atacar si están adyacentes	Implementado
RF-07	Gestionar combate con vida, ataque, defensa y daño no negativo	Implementado
RF-08	Recoger objetos y almacenarlos en inventario propio	Implementado
RF-09	Usar pociones sin confundirlas con equipo	Implementado
RF-10	Equipar arma y escudo desde el	Implementado

ID	Requisito funcional	Estado
	inventario	
RF-11	Activar trampas y registrar el evento	Implementado
RF-12	Cambiar de habitación mediante puertas configuradas	Implementado
RF-13	Bloquear la cámara final hasta obtener la llave necesaria	Implementado
RF-14	Calcular distancia y ruta mínima hacia la salida	Implementado
RF-15	Comprar visualización de ruta con coste de turnos y validación	Implementado
RF-16	Guardar y cargar varias partidas JSON sin sobrescritura obligatoria	Implementado
RF-17	Mostrar registro de eventos de la partida	Implementado
RF-18	Detectar victoria, derrota por vida y derrota por turnos	Implementado

ID	Requisito no funcional	Aplicación en el proyecto
RNF-01	Claridad	La interfaz muestra leyenda, matriz, estado, inventario, acciones y registro de eventos.
RNF-02	Mantenibilidad	Paquetes separados para modelo, lógica, estructuras, persistencia e interfaz.
RNF-03	Pruebas	Tests JUnit para estructuras, lógica, dominio, persistencia y turnos.
RNF-04	Persistencia	JSON legible y compatible con configuraciones de campaña y partidas guardadas.
RNF-05	Restricción EEDD	Uso de estructuras propias en los elementos evaluables, evitando colecciones estándar equivalentes.
RNF-06	Ejecución	Scripts .bat y Maven para compilar, ejecutar y probar el proyecto.

La campaña principal contiene siete habitaciones configuradas en JSON: celda_inicial, pasillo_estrecho, biblioteca_caída, camara_oscura, sala_pinchos, santuario_rojo y camara_del_jefe. También se conservan configuraciones de ejemplo para pruebas de carga y compatibilidad.

3. Metodología y reparto de trabajo

El grupo ha organizado el proyecto por responsabilidades para evitar solapamientos y permitir que cada parte pudiera validarse de forma independiente antes de la integración.

Rol	Integrante	Responsabilidad
Persona 1	Marcos Fernández Pérez	Metodología, requisitos, UML, memoria y pruebas de dominio
Persona 2	Asier López Rodero	Estructuras de datos propias, algoritmos y pruebas de estructuras
Persona 3	Mateo Ramos Gómez	Lógica de juego, JavaFX, persistencia JSON e integración

La planificación se ha seguido en fases: estructuras, modelo, lógica, persistencia, interfaz, pruebas, documentación y revisión final. La integración se ha realizado revisando que cada cambio respetara los requisitos del juego y la restricción de no depender de colecciones estándar equivalentes en las partes evaluadas.

4. Arquitectura del proyecto

El código se organiza en paquetes con responsabilidades separadas. Esta separación facilita pruebas unitarias y permite explicar el sistema en la defensa oral.

Paquete/carpeta	Contenido	Responsabilidad
app	App, Main y rutas	Arranque, menú inicial y localización de recursos
model	Jugador, Enemigo, Objeto, Celda, Habitación, Partida	Modelo de dominio y estado del juego
logic	GestorTurnos, MotorMovimiento, MotorCombate	Reglas de turnos, movimiento y combate
structures	Lista, pila, cola, circular, árbol, grafo y matriz	Estructuras propias utilizadas por dominio y algoritmos
persistence	GestorPersistencia y DTO	Carga de configuración y guardado/carga JSON
ui	VistaPrincipal, paneles, pantallas y pixel art	Interfaz JavaFX y presentación
exceptions	Excepciones del dominio	Errores específicos de movimiento, combate, puertas y configuración
docs	Memoria, UML, diario IA, costes y pruebas	Documentación final y trazabilidad

5. Modelo de dominio y reglas de juego

El modelo distingue entre entidades, objetos, celdas, habitaciones y estado global de la partida. El jugador tiene vida, ataque, defensa, posición, inventario y equipo. Los enemigos tienen vida, ataque, defensa, posición y habitación asignada. Los objetos se diferencian por tipo: arma, escudo, poción o llave.

- La partida se desarrolla por turnos: primero actúa el jugador y después los enemigos vivos de la habitación actual.
- El jugador puede realizar como máximo un movimiento y una acción antes de finalizar el turno.
- El movimiento es ortogonal y de una casilla; se impiden diagonales, muros, límites y celdas ocupadas.
- Los enemigos se aproximan al jugador si lo perciben y atacan cuando están adyacentes.
- Las pociones se usan de forma independiente; arma y escudo se equipan desde la acción de equipar.
- La puerta final requiere una llave de progresión y el sistema informa si el jugador intenta atravesarla sin cumplir el requisito.
- El contador general de turnos limita la partida y la compra de ruta mínima consume turnos, siempre que no provoque derrota inmediata.

6. Estructuras de datos propias y costes

La parte de Estructuras de Datos se apoya en implementaciones propias. Las estructuras no son decorativas: cada una aparece asociada a una necesidad real o a una justificación metodológica del diseño.

Estructura	Uso real en el juego	Justificación	Costes principales
ListaEnlazada	Inventario, listas de habitaciones, enemigos, objetos y adyacencias	Sustituye listas estándar y permite recorrido secuencial propio	agregar $O(n)$, obtener $O(n)$, eliminar $O(n)$
Pila	Soporte de operaciones LIFO y demostración de estructura lineal	Adecuada para invertir recorridos, históricos o acciones deshechas si se amplía	push $O(1)$, pop $O(1)$, cima $O(1)$
Cola	Gestión de turnos y BFS de movimiento/rutas	El orden FIFO modela turnos y exploración por niveles	enqueue $O(1)$, dequeue $O(1)$, consultar frente $O(1)$
ListaCircular	Rotación de entidades/turnos y recorrido cíclico	Representa secuencias repetitivas sin reiniciar índices manuales	avanzar $O(1)$, insertar $O(1)/O(n)$ según posición
Matriz	Celdas de cada habitación	Acceso natural por fila y columna, con habitaciones de tamaños diferentes	get/set $O(1)$, recorrer $O(f*c)$
Grafo	Conexión de habitaciones y camino hacia salida	Cada habitación es un vértice; las puertas son aristas	agregar arista $O(n)$, vecinos $O(n)$, BFS $O(V+E)$

Estructura	Uso real en el juego	Justificación	Costes principales
Árbol	Organización jerárquica de acciones/decisiones	Permite justificar relaciones de decisión y búsqueda por niveles	insertar/buscar $O(h)$, peor caso $O(n)$

La tabla anterior resume la justificación principal. El documento independiente de costes amplía cada operación y se entrega también como anexo separado para facilitar la evaluación de EEDD.

7. Algoritmos principales

El algoritmo más importante es BFS, utilizado en dos contextos: calcular casillas alcanzables dentro de una habitación y obtener caminos mínimos entre habitaciones en el grafo. En ambos casos se emplea una cola propia para conservar la coherencia con la asignatura de Estructuras de Datos.

- BFS en matriz: recorre celdas transitables por niveles y respeta límites, muros y ocupación.
- BFS en grafo: obtiene la secuencia de habitaciones desde la posición actual hasta la habitación de salida.
- Control de turnos: usa una cola de elementos Turnable para mantener orden jugador-enemigos.
- Percepción enemiga: decide si un enemigo se aproxima al jugador y evita movimientos inválidos.
- Cálculo de daño: aplica ataque y defensa, asegurando que el daño final nunca sea negativo.

8. Persistencia JSON

La persistencia se realiza mediante JSON. Existen archivos de configuración para definir campaña, habitaciones, puertas, enemigos y objetos, y también archivos de guardado para conservar partidas completas. El guardado usa selector de archivo para permitir varias partidas y evitar sobrescrituras involuntarias.

- Configuración inicial: define mapa, habitaciones, turnos, entidades y objetos.
- Partida guardada: conserva estado del jugador, habitación actual, inventario, objetos, enemigos y turnos restantes.
- Compatibilidad: se mantienen configuraciones de ejemplo y una campaña completa para pruebas.
- Robustez: los casos de configuración inválida se aíslan en persistencia y excepciones del dominio.

9. Interfaz JavaFX

La interfaz gráfica se ha diseñado para que el estado de la partida sea visible sin consultar la consola. La pantalla principal muestra matriz, leyenda, estado del jugador, inventario, acciones, ruta comprada y registro de eventos.

- La matriz representa jugador, enemigos, objetos, puertas, trampas, salida, muros y suelo.
- Las casillas alcanzables se resaltan, evitando que el jugador interprete como válidos movimientos no permitidos.
- El panel de estado muestra vida, ataque, defensa, habitación, posición, estado y turnos restantes.

- El registro de eventos documenta movimientos, ataques, trampas, puertas bloqueadas, guardados y cargas.
- Los mensajes formales por pantalla ayudan en situaciones relevantes: ruta sin turnos suficientes o puerta final sin llave.

10. Casos de uso

Los casos de uso principales cubren el ciclo completo de partida y las operaciones necesarias para demostrar el proyecto.

Caso de uso	Actor	Precondición	Flujo principal	Postcondición
Mover jugador	Jugador	Partida en curso; no se ha movido en este turno	Selecciona dirección/casilla válida; se actualiza posición y se marcan acciones posibles	Jugador en celda válida; queda disponible una acción o finalizar turno
Atacar enemigo	Jugador	Enemigo adyacente vivo; acción disponible	Se calcula daño; se reduce vida; si muere se retira de la habitación	Acción consumida; después actúan enemigos si procede
Recoger objeto	Jugador	Objeto adyacente o en celda válida; acción disponible	El objeto pasa al inventario y se elimina del tablero	Inventario actualizado y evento registrado
Usar poción	Jugador	Existe poción en inventario; acción disponible	Se aplica curación respetando vida máxima y se consume el objeto	Vida actualizada e inventario sin la poción usada
Equipar	Jugador	Existe arma o escudo en inventario	Se elige elemento; el equipo anterior vuelve al inventario si corresponde	Estadísticas recalculadas
Abrir puerta	Jugador	Puerta adyacente; acción disponible	Se valida destino y requisitos; si procede se cambia de habitación	Nueva habitación cargada o mensaje formal de bloqueo
Comprar ruta	Jugador	Partida en curso; más de tres turnos restantes	Se calcula ruta por BFS y se muestra al jugador	Turnos reducidos y ruta visible; si no hay turnos suficientes se informa
Guardar partida	Jugador	Partida cargada	Se abre selector y se serializa el estado a JSON	Archivo de guardado creado sin sobrescribir obligatoriamente

Caso de uso	Actor	Precondición	Flujo principal	Postcondición
Cargar partida	Jugador	Existe JSON compatible	Se selecciona archivo y se reconstruye estado de partida	Partida recuperada en pantalla

11. Invariantes y contratos

Los invariantes sirven para explicar qué condiciones debe preservar el sistema después de cada operación. También permiten diseñar tests unitarios y detectar errores de integración.

Invariante	Descripción
Posición del jugador	El jugador siempre pertenece a una habitación existente y a una celda dentro de límites.
Vida	La vida de jugador y enemigos nunca baja de 0; una entidad con vida 0 no actúa.
Ocupación de celdas	Una celda no puede contener simultáneamente entidades incompatibles.
Movimiento	No se permite diagonal, salto de más de una casilla, muro o celda ocupada.
Turno	En cada turno el jugador puede realizar como máximo un movimiento y una acción.
Inventario	Los objetos recogidos salen del tablero y no se duplican artificialmente.
Puertas	El cambio de habitación solo se realiza desde una puerta válida y con requisitos cumplidos.
Ruta comprada	La compra no se permite si deja al jugador sin turnos suficientes para continuar.
Persistencia	Guardar/cargar conserva habitación, posición, vida, turnos, inventario, enemigos, objetos y grafo.

Contratos principales: el motor de movimiento no modifica estadísticas de combate; el motor de combate no decide turnos; la persistencia no contiene lógica de juego; la interfaz invoca al controlador pero no implementa reglas de dominio. Esta separación reduce acoplamiento y facilita la defensa del diseño.

12. UML

Los diagramas se mantienen en PlantUML en la carpeta docs/uml y también se incluyen como imágenes. La memoria incorpora los diagramas principales para que el lector pueda revisar diseño y comportamiento sin abrir herramientas externas.

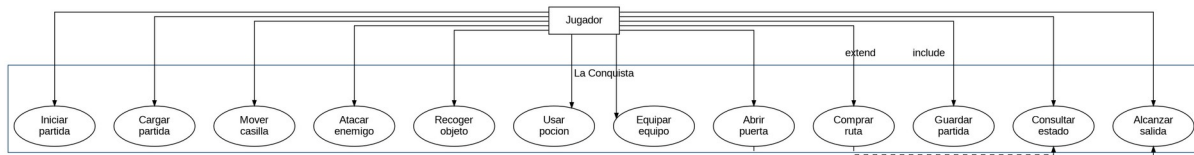


Figura 1. Diagrama de casos de uso.

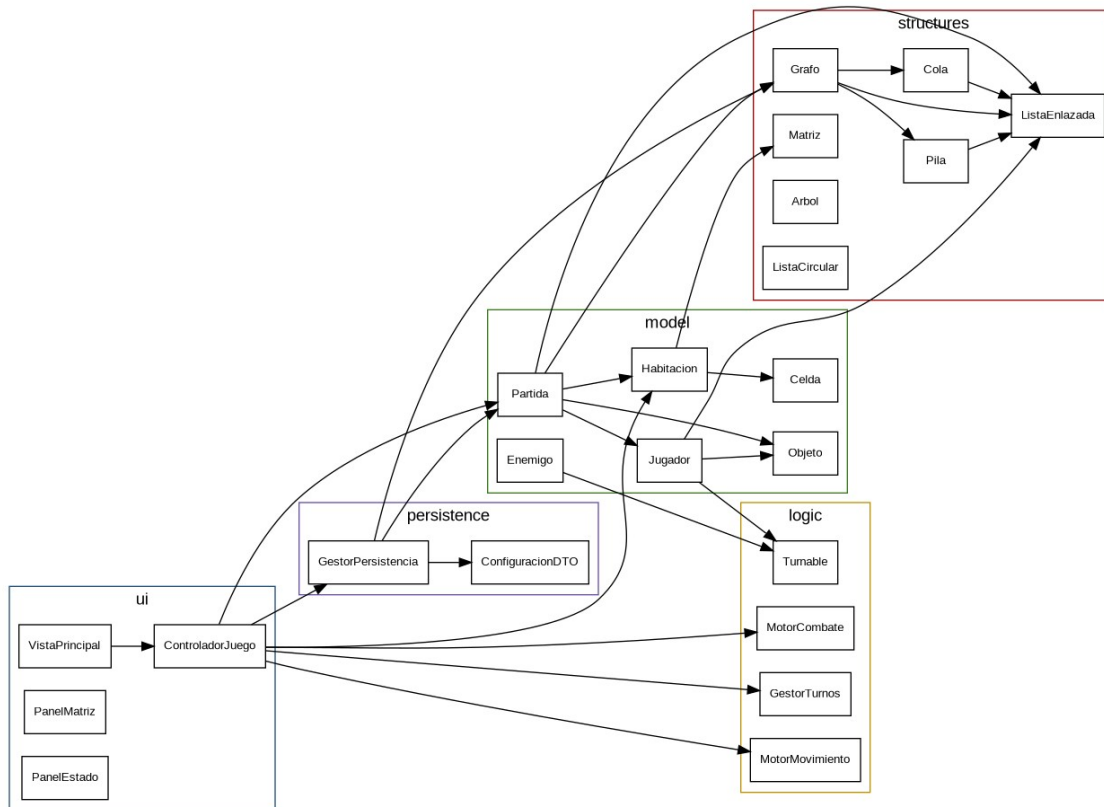


Figura 2. Diagrama de clases.

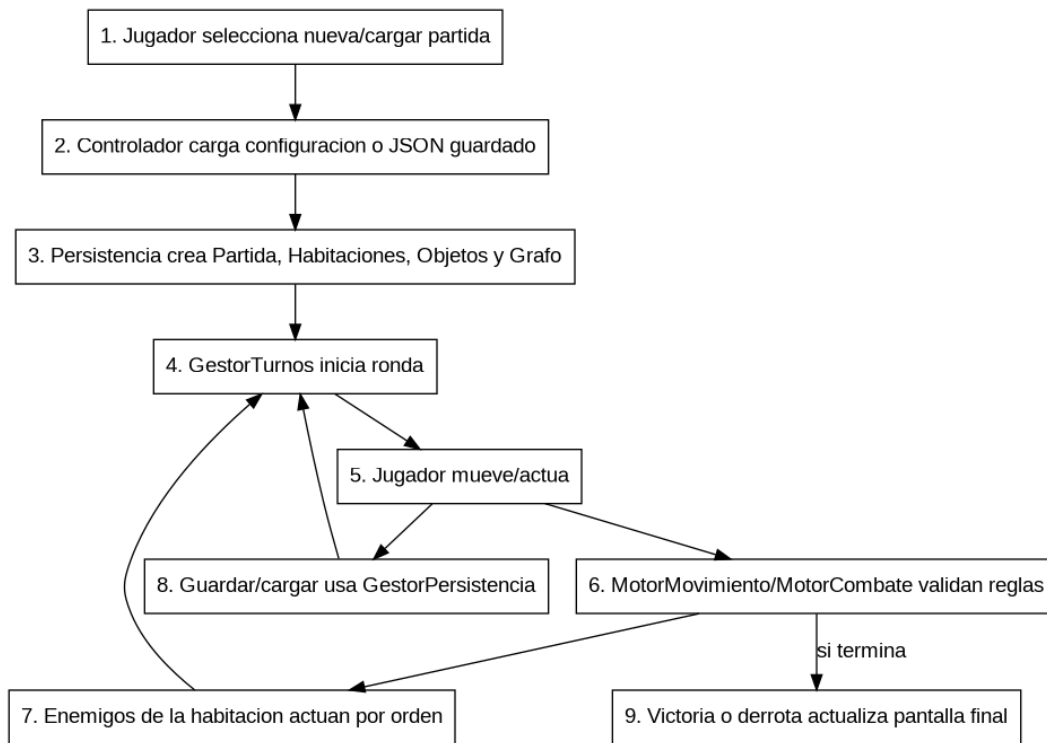


Figura 3. Diagrama de secuencia.

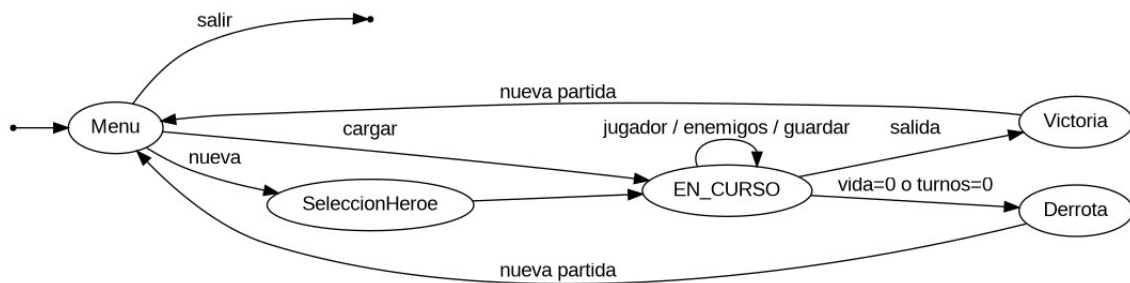


Figura 4. Diagrama de estados.

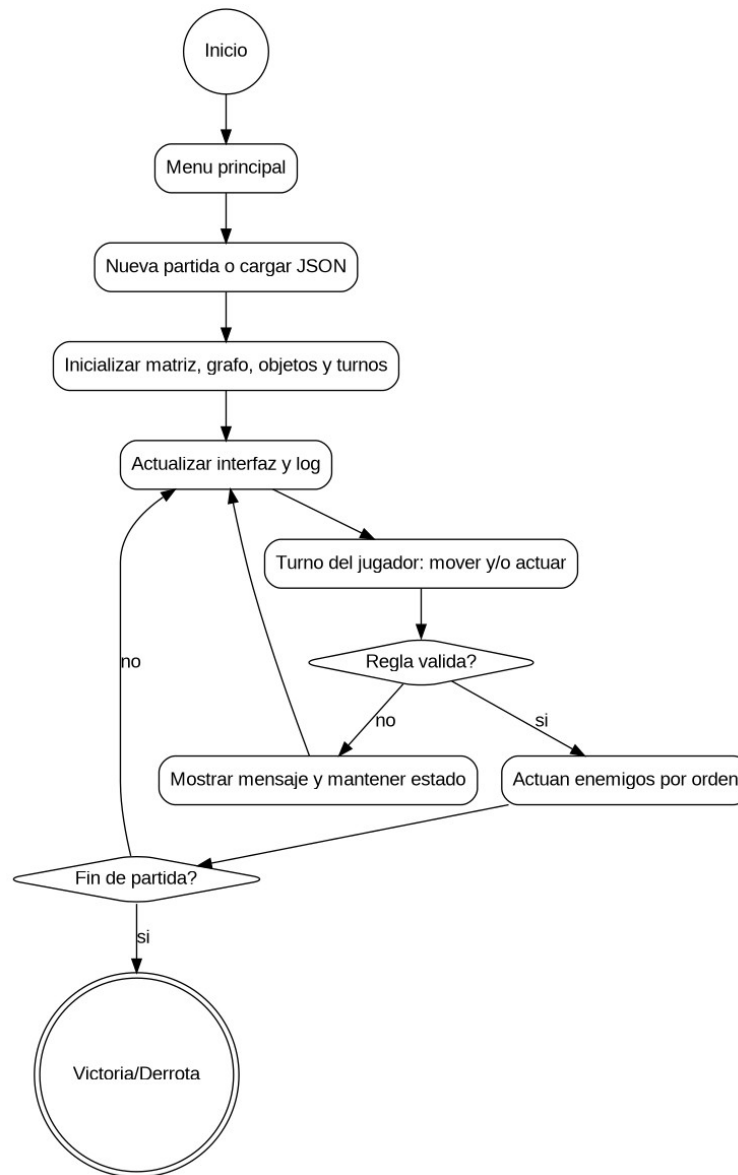


Figura 5. Diagrama de actividad.

13. Pruebas realizadas

Las pruebas se dividen en automáticas y manuales. Las automáticas se han preparado con JUnit 5 para las clases no visuales. Las manuales validan el comportamiento completo de la interfaz JavaFX.

Bloque	Clases o módulos	Qué se comprueba
Estructuras	Lista, pila, cola, lista circular, árbol, grafo y matriz	Operaciones básicas, casos vacíos, recorridos, BFS y acceso a celdas
Dominio	Jugador, partida, objetos y estados	Vida no negativa, curación, equipamiento, inventario y derrota/victoria
Movimiento	MotorMovimiento y controlador	No diagonal, límites, muros, ocupación, movimiento único

Bloque	Clases o módulos	Qué se comprueba
		por turno
Combate	MotorCombate	Daño no negativo, defensa, muerte de enemigos y rangos de daño
Turnos	GestorTurnos y controlador	Jugador primero, enemigos después, un movimiento y una acción por turno
Persistencia	GestorPersistencia	Carga de configuración, guardado, carga y compatibilidad de JSON
Interfaz manual	JavaFX	Ruta comprada, puerta bloqueada, guardado/carga, inventario, victoria y derrota

Ejecución recomendada de pruebas:

test.bat

O de forma equivalente: mvn test

Además, se ha definido una lista de pruebas manuales: iniciar partida, mover, atacar, recoger, usar poción, equipar, activar trampa, comprar ruta, intentar comprar ruta sin turnos, abrir puerta sin llave, guardar varias partidas, cargar una partida, ganar y perder.

14. Uso de IA

El proyecto se ha desarrollado con apoyo de un agente de IA identificado en la documentación como Opencode. La IA se ha utilizado como apoyo supervisado para generar propuestas, revisar errores, documentar decisiones y acelerar iteraciones. Las decisiones finales se han validado por el grupo y el diario completo se entrega como documento independiente.

- Uso aceptado: apoyo en implementación, depuración, revisión de requisitos, generación de documentación y tests.
- Control humano: las mecánicas se probaron manualmente y se corrigieron cuando no cumplían el enunciado o no resultaban jugables.
- Trazabilidad: el diario registra objetivo, prompt resumido, resultado, cambios aceptados y cambios rechazados.
- Crítica: el uso de IA permite iterar rápido, pero exige revisión humana para evitar incoherencias de reglas, documentación o interfaz.

15. Crítica del proyecto

La crítica del proyecto se plantea como una revisión técnica del resultado final y no como una simple enumeración de fallos. El objetivo ha sido valorar si la solución cumple el enunciado, si las decisiones tomadas son defendibles y qué aspectos podrían mejorarse en una evolución posterior del sistema.

El balance general es positivo. La aplicación final permite jugar una partida completa, visualizar habitaciones como matrices, recorrer un mapa de habitaciones conectado mediante grafo, gestionar turnos, enemigos, inventario, objetos, puertas, trampas, ruta mínima y guardado/carga en JSON. Además, la implementación se apoya en estructuras propias y en pruebas de las partes no visuales, lo que conecta la práctica con las dos asignaturas.

Durante el desarrollo se comprobó que una versión aparentemente funcional no siempre era correcta desde el punto de vista metodológico. Algunos errores detectados fueron especialmente útiles: movimientos que cerraban el turno demasiado pronto, enemigos que atacaban en momentos poco justos, guardados que podían sobrescribir partidas, uso ambiguo de objetos equipables y mensajes de interfaz insuficientes. La corrección de estos problemas mejoró el cumplimiento del requisito de un movimiento máximo y una acción máxima por turno, y reforzó la claridad de la interfaz.

Desde el punto de vista de Estructuras de Datos, el proyecto permite justificar varias estructuras: listas para colecciones del dominio, cola para BFS y turnos, matriz para representar cada habitación, grafo para las conexiones entre salas, lista circular para recorridos cíclicos y árbol para organizar decisiones/acciones. La principal dificultad fue evitar que las estructuras quedaran como elementos aislados: por ello se documenta qué problema resuelve cada una y qué coste tiene en operaciones reales del juego.

Desde el punto de vista de Metodología de la Programación, la mayor mejora fue separar responsabilidades: modelo, lógica, persistencia, interfaz, excepciones y pruebas. Esta separación reduce el acoplamiento y hace que el proyecto sea más explicable. También facilita defender por qué una regla pertenece a la lógica de juego y no a JavaFX, o por qué la persistencia JSON debe limitarse a guardar y recuperar estado sin decidir reglas de partida.

La interfaz no busca competir con un videojuego comercial, sino ser clara. Esta decisión es coherente con el enunciado, que valora más la correcta representación de la matriz, el estado del jugador, las acciones y el registro de eventos que el uso de gráficos complejos. Aun así, sería una mejora futura sustituir las letras y colores por iconos o sprites, siempre manteniendo la misma lógica interna.

Las limitaciones principales son razonables para el alcance de una práctica de primer curso. La inteligencia de los enemigos es sencilla, la campaña es limitada, la llave maestra se ha diseñado como objeto de progresión y no como un sistema complejo de llaves múltiples, y el combate mantiene una fórmula simple basada en ataque, defensa y aleatoriedad. Estas decisiones reducen complejidad accidental y permiten concentrar la evaluación en diseño, estructuras, persistencia y pruebas.

Como mejoras futuras, se podrían añadir más habitaciones, enemigos con distintos patrones, equipamiento más avanzado, dificultad configurable, mejor presentación gráfica, validación más estricta de todos los JSON de entrada, pruebas de integración automatizadas y una pantalla final con

estadísticas de la partida. También sería posible ampliar el árbol de acciones para modelar decisiones más complejas y aprovechar más esa estructura en tiempo de ejecución.

La conclusión crítica es que el proyecto ha evolucionado desde una implementación funcional hacia una solución más coherente con el enunciado. Los fallos detectados no se ocultaron, sino que se usaron como parte del proceso de mejora: se corrigieron reglas de turnos, guardado, carga, interfaz, mensajes y documentación. Por tanto, el resultado final es defendible tanto por funcionamiento como por metodología.

16. Ejecución y entrega

Para ejecutar el juego en Windows se entra en la carpeta Practica_Final y se ejecuta:

run.bat

Para ejecutar las pruebas:

test.bat

Entregables recomendados: código fuente, memoria PDF, diario de IA, costes de estructuras, UML en PlantUML e imágenes, archivos JSON de configuración, tests y vídeo explicativo del grupo.